Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Building lists from lists

Today
Welcome to your next lesson: Building lists from lists.

I'm preparing your personalized lesson based on what you've already learned, what's coming next, and the best way for you to keep building your skills.

Step by step, you're moving forward.

Imagine you have a list of exam scores:

Python

```
scores = [45, 82, 99, 61, 37]
```

Now you want a new list that only has the passing scores (let's say 60 and above). You don't change scores; you build a new list from it. That pattern is what this lesson is about.

Quick check, Ron 👇

Does "build a new list from an old list" feel:

more like: "change the original list directly", or
more like: "leave the original alone, make a fresh list based on it"?

You can just answer in your own words, no need to be formal.

leave orignal alone

Perfect—that's exactly the idea ☑

We leave the original alone and build a fresh list based on it.

Let's look at a very simple version of this pattern step by step:

Python

```
scores = [45, 82, 99, 61, 37]

passing_scores = []          # 1️⃣ start with an empty result list

for score in scores:         # 2️⃣ loop over the original list
    if score >= 60:          # 3️⃣ decide if we want to keep this one
        passing_scores.append(score)   # 4️⃣ append to the new list

print(passing_scores)
```

Visually:

scores (original): [45, 82, 99, 61, 37]
passing_scores (new): starts as [], then grows: [82] → [82, 99] → [82, 99, 61]
Does this step-by-step flow make sense so far, or is there any line you'd like me to slow down and unpack more (for example, passing_scores = [] or the append part)?

yes

Nice 👍

Let's zoom in on one key idea in that code: the "accumulator" list.

Accumulator list = a new list that starts empty ([]) and slowly grows as you append items inside the loop.

In our example:

Python

```
passing_scores = []      # accumulator: starts empty

for score in scores:
    if score >= 60:
        passing_scores.append(score)   # accumulator grows here
```

So the 3 core steps are:

Create an empty list (the accumulator).
Loop over the original list.
Decide which items to append into the accumulator.
Quick check: are you comfortable with this idea of "start with [], then append inside the loop to build a new list"? Or should I walk through a smaller, simpler example (like building a list of even numbers) before you try one yourself?

I'm comfortable

Great, then we can level it up a bit 💪

Now I want to show you one specific pattern: using a loop to filter a list.

Filter pattern = go through a list and keep only items that match a condition, putting those items into a new list.

Example with numbers:

Python

```
numbers = [3, 10, 15, 8, 21, 4]

evens = []                 # accumulator

for n in numbers:
    if n % 2 == 0:         # condition: is n even?
        evens.append(n)    # keep it

print(evens)               # [10, 8, 4]
```

Here we:

don't change numbers

do build evens by filtering out only the even ones
Before I ask you to code, I want to be sure this idea is clear:

Does this feel similar to the passing-scores example, just with a different condition (n % 2 == 0 instead of score >= 60)?

Yes, it's similar

Nice 👌 that’s exactly the connection I wanted you to see.

Now let’s flip the idea a bit.

So far, we’ve been filtering (keeping some original items).

Another common pattern is transforming: we still build a new list, but we change each item before we append it.

Example: you have prices, and you want a new list with a 10% discount applied:

Python

```
prices = [100, 250, 80]

discounted = []                 # accumulator

for price in prices:
    new_price = price * 0.9     # transform step
    discounted.append(new_price)

print(discounted)               # [90.0, 225.0, 72.0]
```

Visually:

original: [100, 250, 80]
new: [90.0, 225.0, 72.0] (same length, but values changed)

Here we’re not deciding “keep vs skip”; we’re deciding “how to change it before appending”.

Does the difference between filter (keep some items) and transform (change items) feel clear with this example, or should I draw a small side□by□side comparison next?

It's clear now

Awesome 🌟 Since filter vs transform is clear, let’s get you hands-on.

We’ll start with a filter exercise.

Here’s the setup:

Python

```
temperatures = [55, 72, 49, 90, 66, 59, 80]
```

Goal:

Build a new list that contains only the temperatures that are 70 or higher (hot days).

Step-by-step pattern you should use in the code editor:

Create an empty list, like hot_days = [].
Loop over temperatures.
If a temperature is 70 or more, append it to hot_days.

At the end, print(hot_days).

👉 Please open the code editor, write this program in Python, and then hit Submit code (not just Run).

Once you submit, I'll check it and we'll build on it.

Python

```
temperatures = [55, 72, 49, 90, 66, 59, 80]
hot_days = []

for temp in temperatures:
    if temp >= 70:
        hot_days.append(temp)
print(hot_days)
```

Nice work, Ron 🎯

Your code uses the accumulator pattern perfectly:

Python

```
hot_days = []          # start empty
for temp in temperatures:
    if temp >= 70:     # filter condition
        hot_days.append(temp)
```

Output [72, 90, 80] is exactly right ☑

Now I want to show you one more important idea:

we can separate the condition from the loop using a helper function, to keep things clean and readable.

Example:

Python

```
def is_hot(temp):
    return temp >= 70

hot_days = []
for temp in temperatures:
    if is_hot(temp):
        hot_days.append(temp)
```

Here:

is_hot holds the logic for "what counts as hot"
the loop just says: "if this is hot, append it"
Does this separation (helper function + loop) make sense so far, or should I slow down and unpack the def is_hot(temp): part more?

It makes sense

What do you think? Pick the option that really builds a new list from an old list 👇

In which of these snippets is the accumulator pattern (building a new list from an old list) used correctly?

A.

```
nums = [1, 2, 3]
for n in nums:
    if n > 1:
        print(n)
```

B.

```
nums = [1, 2, 3]
result = []
for n in nums:
    if n > 1:
        result.append(n)
C.

nums = [1, 2, 3]
nums.append(4)
D.

nums = [1, 2, 3]
result = []
print(result)
```

Yes! 🎉 You picked the right one.

This snippet:

Python

```
nums = [1, 2, 3]
result = []
for n in nums:
    if n > 1:
        result.append(n)
```

is a perfect example of the accumulator pattern because:

result = [] → starts as an empty list (the accumulator)
for n in nums: → loops over the original list
if n > 1: → applies a condition
result.append(n) → builds a new list from selected items

The other options either just print, change the original list, or never add anything to the new list.

You've got a solid grip on this pattern 💪

Next, I'd like you to use a helper function with this pattern in your own code. Ready for a small exercise using def + accumulator?

I'm ready!

Love that 🔥

Let's do a small, real‑world style task with a helper function + accumulator.

You have this list:

Python

```
names = ["Anna", "Bob", "Christopher", "Eve", "Jonathan", "Li"]
```

Goal: build a new list that contains only short names (let's say length ≤ 3).

Step by step (in the code editor):

Define a helper function, e.g.:

```
def is_short(name):
    # return True if the name is 3 chars or less
```

Create an empty list, like short_names = [].

Loop over names.

Inside the loop, use your helper: if is_short(name): ...

If it’s short, append it to short_names.

At the end, print(short_names).

👉 Please write this in the code editor and hit Submit code. I’ll check your solution and we’ll build from there.

Python

```
names = ["Anna", "Bob", "Christopher", "Eve", "Jonathan", "Li"]
short_names = []

def is_short(name):
  return len(name) <= 3

for name in names:
    if is_short(name):
        short_names.append(name)
print(short_names)
```

Beautifully done, Ron 🎯

Your code nails all the key ideas:

```
names = ["Anna", "Bob", "Christopher", "Eve", "Jonathan", "Li"]
short_names = []

def is_short(name):
    return len(name) <= 3   # clean, single-purpose helper ☑

for name in names:
    if is_short(name):      # loop just asks the helper
        short_names.append(name)  # accumulator pattern

print(short_names)          # ['Bob', 'Eve', 'Li']
```

You:

Used an accumulator list (short_names) and built it with append.
Filtered the original list based on a condition.
Separated logic (is_short) from assembly (the loop + append).
That covers everything in this lesson’s goals 👌

You’ve fully mastered building lists from lists:

filtering
transforming
and keeping loop logic clean with helpers.
From here, the next lessons will build on this pattern and make your data-processing code more powerful and clear.

Whenever you’re ready, you can jump to the next lesson in the Learn tab. And if you want extra practice on this pattern later, just tell me “more accumulator exercises” and I’ll spin some up for you.

```
temperatures = [55, 72, 49, 90, 66, 59, 80]
hot_days = []

for temp in temperatures:
    if temp >= 70:
        hot_days.append(temp)
print(hot_days)

names = ["Anna", "Bob", "Christopher", "Eve", "Jonathan", "Li"]
```

```
short_names = []

def is_short(name):
  return len(name) <= 3

for name in names:
    if is_short(name):
        short_names.append(name)
print(short_names)

```